Informàtica 1

Semester 2026-27 Q1

Airline Management

Project statement



Introduction

As many other companies, airlines need to manage their resources to produce their services. The main resource of an airline is its fleet of aircraft. The main services an airline offers are its flight operations. Selling the flight tickets provides the incomes to the company, so the company would like to maximize the number of operations. The used resources, such as the fleet, are expensive and the company will try to optimize the use of its aircrafts. In this project you will develop an application to help a company to obtain as much benefit as possible.

Given the flight schedule for a specific day and the list of aircraft available for that day, the application should allow the user to assign flights to aircraft using different assignment algorithms. It should also allow the user to analyze the company's profit from each assignment algorithm. Finally, it should display the results in clear and concise graphs.

During the process you must deliver four versions of the tool, of increasing complexity. This document describes the requirements for each version. The first version will be done in a very guided way, step by step. The following versions will not be so guided, so that you can train your abilities to plan work, distribute tasks, integrate partial results, etc. In the last version you will put your initiative and creativity into play to incorporate functionalities into the tool that surprise us.

The description of each version includes quality criteria that teachers will use to determine what has been done well and what has not been done well in that version. These criteria should be a

useful guide for you to ensure that you are developing the project well. Take into account that quality criteria are incremental. That means that que quality criteria for a given version include all the quality criteria for previous versions and additional quality criteria for that new version.

Each of the versions will be evaluated by teachers, using the corresponding quality criteria. The evaluation of the first three versions will be purely formative. That is, the teachers will not give any grade, but they will give you comments that should help you improve for the next version. Only the latest version will be graded (the quality criteria for that version will indicate how it will be graded). Remember that this grade will be the same for all members of the group and that it has a weight of 30% in the course grade.

In the subject planning that appears in Atenea, the delivery date of each of the versions is clearly indicated. These delivery dates are immutable. There is no excuse for not delivering the corresponding version on time because since it does not have an associated grade, what is not operational in one version is pending for the next. The important thing is that teachers should be able to evaluate each version when planned and make relevant comments to each team as soon as possible.

Each delivery consists of the developed code and an explanatory video. The code must be delivered in the form of a repository on GitHub, which is a free tool that allows code storage in the cloud and management of versions (exactly what you need for this project). In Atenea it will only be necessary to provide the URL of the GitHub repository that contains your code, which naturally must be publicly accessible.

The video must show in no more than 5 minutes that the tool meets the corresponding requirements. It should also show the parts of code that were added to the previous version, with relevant explanations, including the actions taken by the team as a result of the feedback given by teachers for the previous version. For example, it will be positively valued if external help (from other colleagues or from AI tools such as Copilot or ChatGPT) to incorporate some parts of code has been clearly explained in the video. The link to the video must be included in the README of the project repository in GitHub, together con a formal declaration of use AI tools in the development of every version.

It is important to note that teachers will conduct weekly follow-up meetings in class. Through these meetings, they will monitor the progress of the work, verify that all team members are making their assigned contributions, and ensure that each member can explain and justify their contributions in detail, even if he/she has used AI tools during the development. The results of this follow-up will be taken into account in the final evaluation of every student.

Version 1

The first version of the tool will only deal with the management of information about airlines. An airline has a name, a fleet (a list of aircrafts) and a list of operations. We need an application with a nice user interface that allows the user to load the information from text files and show this information y text boxes and plots.

The work is organized in 9 steps described below. It is important to read the instructions and observations of each step carefully.

It is strongly recommended that to develop this first version you work together, side by side, which will facilitate shared learning of those topics that you have studied separately. Ideally, each team member would develop his own version of the application with the support of teammates, to practice with basic concepts. The time will come, in future versions, for you to test your ability to separate tasks, distribute them equitably and then harmoniously integrate the results.

Step 1

Create the file *aircraft.py* with the definition of the *Aircraft* class. The class must have the following attributes and the corresponding constructor.

- Callsign (a text)
- Type (a text)
- Capacity (an integer)

Add to *aircraft.py* the following function:

```
PrintAircraft (aircraft)
#Prints in console the data of the aricraft
```

Step 2

Create a file *test_aircraft.py* the with following code:

```
from aircraft include *
aircraft = Aircraft ("AAAA", “A220-100”, 135)
PrintAircraft (aircraft)
```

This code will allow you to check if the definitions in *aricraft.py* are ok. Both files must be in the same folder. You only must run the *test_ aricraft.py* code and check if the results are correct. Try with different aircraft data.

Step 3

Create the file *flight.py* with the definition of the *Flight* class, that contains the information of an operation. The class must have the following attributes and the corresponding constructor.

- ID (a text)
- DepartureAirport (a text with the ICAO code of the airport)
- ArrivalAirport (a text with the ICAO code of the airport)
- DepartureTime (a text in the format HH:MM)
- ArrivalTime (a text in the format HH:MM)
- Passengers (an integer)

Add to *flight.py* the following functions:

```
PrintFlight (flight)
#Prints in console the data of the flight

TimeToMinutes (time)
# Converts and returns time hh:mm to minuts counted from 00:00

FlightDuration (flight)
# Returns the duration in minutes
```

Step 4

Add also to *flight.py* the following code:

```
# test section
if __name__ == "__main__":
       flight = Flight ("A11", "LEBL", "EGLL", "12:30", "14:30", 135)
       PrintFlight (flight)
       print (FlightDuration (flight))
```

This code can be used to test the functions defined in *flight.py*. You just need to run the *flight.py* script and check the result. The system will detect that you are running *flight.py* as a main program and will execute the code in this test section. Using this approach you do not need to prepare a separate script to test the definitions as you did in step 2. So you may want to remove the file *test_aircraft.py* and add a text section in *aircraft.py*.

Step 5

Create the file *airline.py* with the definition of the *Airline* class. The class must have the following attributes and the corresponding constructor.

- Name (a text)
- Fleet (a list of Aircraft)
- Operations (a list of Flight)

Add to *airline.py* the following function:

```
PrintAirline (airline)
#Prints in console the data of the airline
```

Add also to *airline.py* the following test section:

```
# test section
if __name__ == "__main__":
       from aircraft include *
       from flight include *
       aircrafts = []
```

```
aircrafts.append (Aircraft ("AAAA", "A220-100", 135))
aircrafts.append (Aircraft ("BBBB", "A230-100", 200))

flights = []
flights.append (Flight ("A11", "LEBL", "EGLL", "12:30", "14:30"))
flights.append (Flight ("A12", "EGLL", "LEBL", "16:30", "18:30"))

airlisne = Airline ("Vueling", aircrafts, flights)
printAirline (airline)
```

Step 6

The data of aircrafts for an airline is stored in text files. We need functions to load/save info in/to the file. Add the functions with the following headers to *aircraft.py*:

```
LoadAircrafts (filename)
'''Opens the file with name received as parameter and with the format
described below (two lines shown in the example, one line per aircraft) and
returns a list of aircraft with the data found in the file. If the file does
not exist the function returns an empty list. Ignore file lines that do not
have the expected format.

AAAA A220-100 135
BBBB A230-100 200

'''

SaveAircrafts (aircrafts, filename)
'''Saves in the file with name received as parameter the data from the list of
aircrafts, in the same format as described above.

'''
PrintAircrafts (aircrafts)
'''Prints in console the data of all the aircrafts in the list

'''
AddAircraft (aircrafts, aircraft)
'''Adds the aircraft to the list of aircrafts.
'''
RemoveAircraft (aircrafts, callsign)
'''Remove from the list of aircraft the aircraft with the callsign
received as parameter. If the aircraft is not in the list an error
code is returned.
'''
```

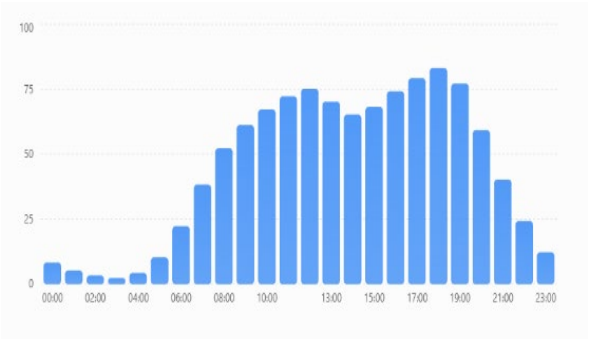
Add to the test section of *Aircraft.py* the code required to test the new functions. To that purpose you can use the text file with aircraft data that you will find in Atenea.

Step 7

The data of operations for an airline is also stored in text files. So, we need to include in *flight.py* new functions to load, save, print, add and remove flights, exactly as you did for aircrafts. Define de functions you need, include the code in *flight.py* and include also in the test section the code you need to test the new functions. To that purpose you can use the text file with flight data that you will find in Atenea.

Modify the test section developed in step 5 so that the data for aircrafts and operations of the airline are loaded from the corresponding text files.

Step 8



Let's include some graphics that might make the application more user-friendly. Add to *airline.py* one function to create a bar plot to show the number of flights that would be in the air in every one-hour period, from 00:00 to 23:00. Include in the test section the code required to test this function.

```
PlotFlights (airline)
'''Shows a bar plot with the number of flights on air in every hour period
'''
```

Step 9

Create the *interface.py* file a graphical interface in tkinter that allows you to perform, in the most user-friendly way possible, all the operations implemented in *airline.py*.

Quality criteria for version 1

Correct • All code files have been developed (*aircraft.py, flight.py* and *airline.py* and *interface.py*). • The test section in every python file allows you to verify the correct functioning of all the functions associated with every class. • Through the graphical interface the user can: o Load, show, add, remove and save aircrafts o Load, show, add, remove and save flights o Plot the number of flights in every hour period
Video • The video makes it clear which functionalities are operational among those planned for this version. • The video shows the most relevant pieces of code.
Robust • The application resists without blocking the errors that the user may make when interacting with it. It never crashes. • The application detects and reports errors when reading files.
Friendly • The user has no doubt when interacting with the graphical interface. • The graphical interface is visually attractive.
Organized and documented • The code is well structured and indented. It is easy to locate where in the code every functionality is implemented.

• Clarifying comments have been added.
• The github repo is well organized: includes the developed code and a Readme with the name (and a picture) of the team members, the link to the video and a tag for the version. The repo also includes a declaration of how IA tools have been used in the development of the version.